COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010	**SECTION:**	CPE21S2
DATE PERFORMED:	8/5/26	**DATE SUBMITTED:**	8/5/26
aNAME:	SUBALISID, JAPPETH ROYCE B.	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. ___	**[ACTIVITY NAME]**

1. PROCEDURE OUTPUT

ILO A: Testing the queue STL

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> cue);

int main() {
        //create an object
    std::queue<char> myQ;

    //use the enqueue operation
    myQ.push('R');
    myQ.push('O');
    myQ.push('Y');
    myQ.push('C');
    myQ.push('E');

    std::cout << "Front: " << myQ.front() << std::endl;
    std::cout << "Back: " << myQ.back() << std::endl;
    std::cout << "Whole: ";
    display(myQ);
    //use the dequeue operation
    myQ.pop();
    display(myQ);
    myQ.pop();
    display(myQ);
    myQ.pop();
    display(myQ);
    myQ.pop();
    display(myQ);
    myQ.pop();
    display(myQ);

    return 0;
}
```

Main Function

- Push, pop, front, back, and empty functions work as expected and allowed me to create a custom display all function

```
void display(std::queue<char> cue) {
    if(cue.empty()){
        std::cout << "Queue is empty\n";
    } else {
    std::queue<char> copyCue = cue;
    while(!copyCue.empty()) {
        std::cout << copyCue.front();
        copyCue.pop();
    }
    std::cout << std::endl;}
}
```

```
Front: R
Back: E
Whole: ROYCE
OYCE
YCE
CE
E
Queue is empty
```

Display All Function

- Created using the empty() function and printing the front, then popping, until the queue is empty

ILO B: Creating Queue Header file using Linked List

```
#include <iostream>
#include "queueLL.h"

int main() {

    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    enqueue('R', front, back);
    enqueue('O', front, back);
    enqueue('Y', front, back);
    enqueue('C', front, back);
    enqueue('E', front, back);
    std::cout << "\nTesting if enqueue workedS";
    std::cout << "\nFront: " << front->data << "\tBack: " << back->data;

    std::cout << "\nTesting if dequeue worked";
    dequeue(front, back);
    std::cout << "\nFront: " << front->data << "\tBack: " << back->data;

    std::cout << "\nDisplaying all\n";
    displayAll(front);

    std::cout << "\nTesting is empty\n";
    std::cout << (isEmpty(front, back)? "Queue is empty\n" : "Queue is not empty\n");
    std::cout << "\nDeleting all\n";
    dequeue(front, back);
    dequeue(front, back);
    dequeue(front, back);
    dequeue(front, back);
    std::cout << (isEmpty(front, back)? "Queue is empty\n" : "Queue is not empty\n");

    return 0;
}
```

Main Function

```
#ifndef QUEUELL_H
#define QUEUELL_H

//class node
template<typename A>
class qNode{
    public:
        A data;
        qNode* next;
};

//new node creator
template<typename A>
qNode<A>* createNode(A newData) {
    //allocate spcace
    qNode<A>* newNode = new qNode<A>;
    //store new data
    newNode->data = newData;
    //point newnode to null
    newNode->next = nullptr;

    return newNode;
}
```

Node Constructors

- Node class, which is a singly linked list type and a node constructor that takes in a data as an argument

```
//enqueue
template<typename A>
void enqueue(A newData, qNode<A>*& frontPtr, qNode<A>*& backPtr) {
    //create new Node
    qNode<A>* newNode = createNode(newData);
    //check if empty
    if(frontPtr == nullptr && backPtr == nullptr){
        frontPtr = newNode;
        backPtr = newNode;
        return;
    }
    //insert at the back
    backPtr->next = newNode;
    backPtr = newNode;
}
```

Enqueue Function

- Checks if queue is empty then assigns the new node accordingly

```cpp
//dequeue
template<typename A>
void dequeue(qNode<A>*& frontPtr, qNode<A>*& backPtr) {
    //check if queue is empty
    if(frontPtr == nullptr && backPtr == nullptr) {
        std::cout << "\nThe queue is empty\n";
        return;
    }
    //create a temp var to store node to be deleted
    qNode<A>* delTemp = nullptr;
    //assign front to temp
    delTemp = frontPtr;
    //check if only one element
    if(frontPtr->next == nullptr && backPtr->next == nullptr){
        frontPtr = nullptr;
        backPtr = nullptr;
        delete delTemp;
        return;
    }

    frontPtr = frontPtr->next;
    delTemp->next = nullptr;
    delete delTemp;
}
```

Dequeue Function

- Checks if the queue is empty, then deletes the front node if not

```cpp
//display all
template <typename A>
void displayAll(qNode<A>*& front) {
    qNode<A>* temp = front;
    while(temp != nullptr) {
        std::cout << temp->data << std::endl;
        temp = temp->next;
    }
}

//isempty
template <typename A>
bool isEmpty(qNode<A>*& frontPtr, qNode<A>*& backPtr) {
    if(frontPtr == nullptr && backPtr == nullptr) return true;
    return false;
}

#endif
```

DisplayAll and IsEmpty Functions

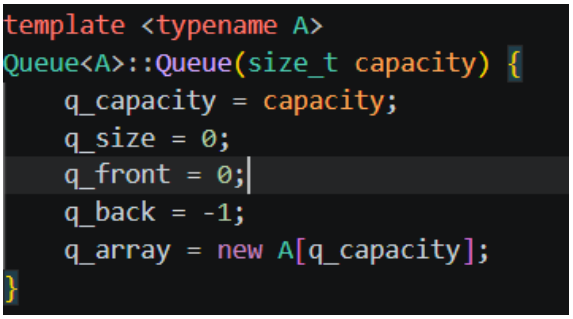
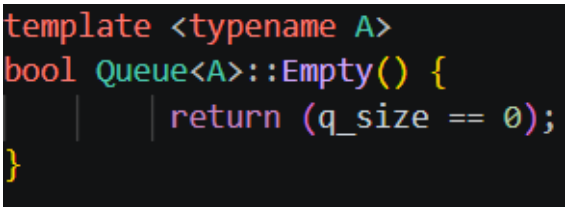
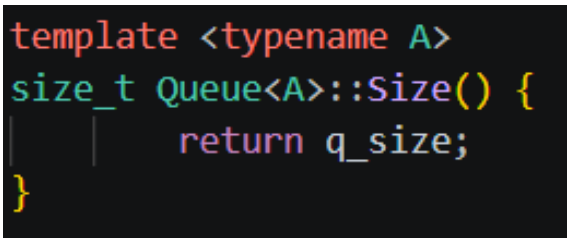
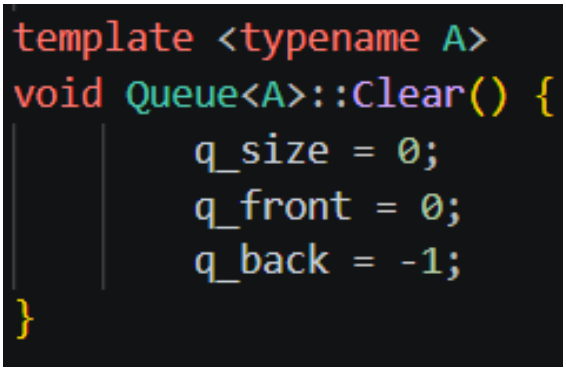
- Display all is the same as in singly linked list where we create a temp pointer to not disturb the original nodes.
- Is empty checks if the front and back are both null, therefore having no queue content

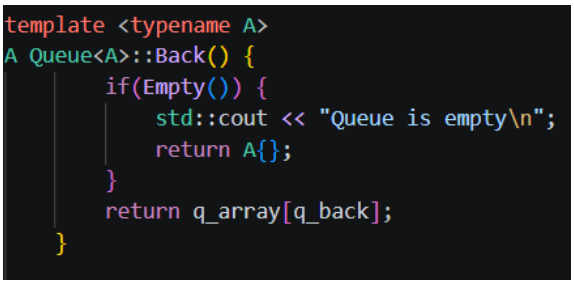
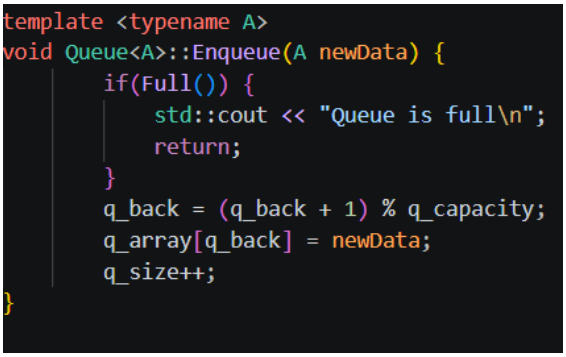
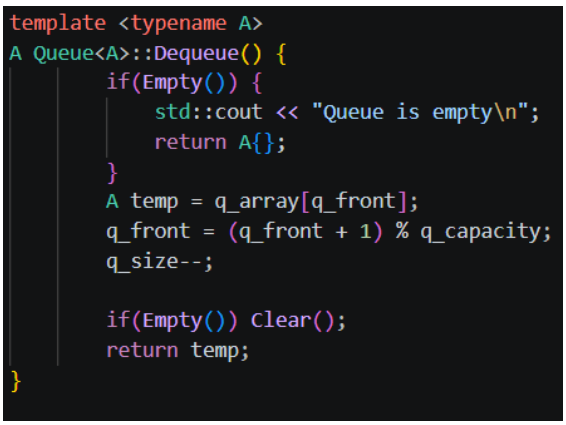
ILO B.2. Array-Based Implementation

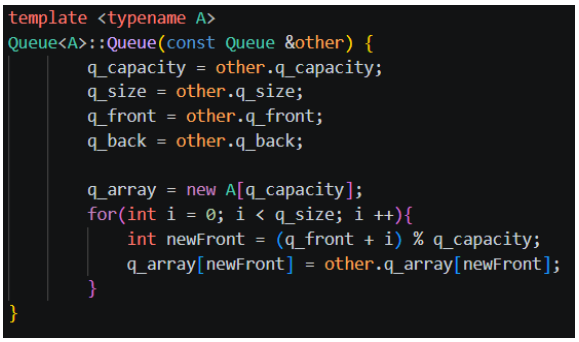
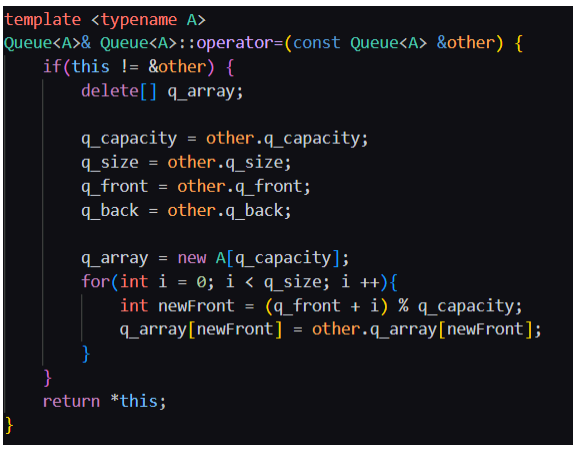
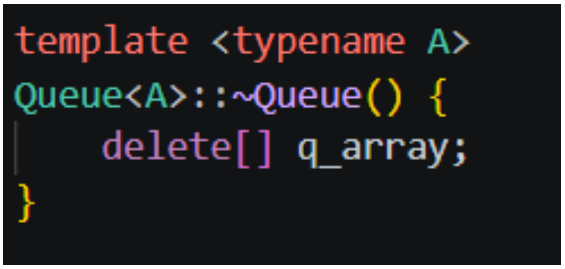
In C++, we could implement an array-based queue as a class. To conserve space, we'll implement it as a "circular queue", an array in which the last position is logically connected back to the first position to make a circle. This is sometimes also called a "ring buffer".

Member Functions

The example functions described here correspond to the interface of the queue class in the C++ standard library.

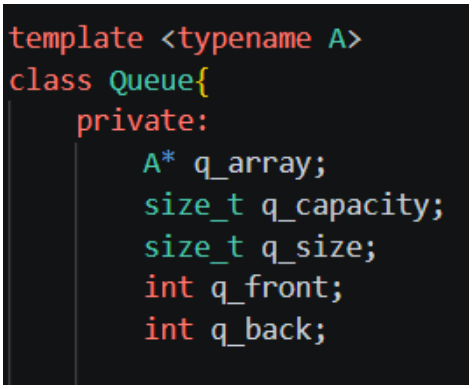
Operation	Description	Code
Queue() (Constructor).	Constructs an empty queue	
Empty()	Tests whether the queue is empty. Returns true if the queue contains no elements; otherwise returns false.	
Size()	Returns the number of elements currently stored in the queue.	
Clear()	Resets the queue size to 0 and restores q_front and q_back to their initial values. Does not deallocate dynamic memory or change the queue capacity.	

Front()	Returns the element at the front of the queue (the next element to be removed).	
Back()	Returns the element at the back of the queue (the most recently added element).	
Enqueue()	Inserts a new element at the back of the queue.	
Dequeue()	Removes the element at the front of the queue.	

Copy Constructor	Creates a new queue as a copy of an existing queue. Unlike vectors or array-based stacks, queue elements may wrap around the array, so the entire queue array must be copied.	
Copy Assignment Operator	Replaces the contents of one queue with a copy of another queue. Since queue elements may not occupy consecutive positions, the entire queue array must be copied.	
Destructor	Deletes the dynamically allocated queue array and releases its memory.	

Your Tasks:

- Create a queue class that will contain the array.



- Operations and member functions described in this section must be in your developed code.

```
    public:
        Queue(size_t capacity = 10);
        ~Queue();
        Queue(const Queue &other);
        Queue& operator=(const Queue &other);

        bool Empty();
        bool Full();
        size_t Size();
        void Clear();
        A Front();
        A Back();
        void Enqueue(A newData);
        A Dequeue();
        void DisplayAll();
};
```

- Test the functions and show the output

```cpp
int main() {
    Queue<char> q(5);
    std::cout << "Enqueueing: ROYCE\n";
    q.Enqueue('R');
    q.Enqueue('O');
    q.Enqueue('Y');
    q.Enqueue('C');
    q.Enqueue('E');

    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;

    std::cout << "Queue: ";
    q.DisplayAll();

    Queue<char> r = q;
    std::cout << "Copied Array: \n";
    r.DisplayAll();

    Queue<char> s;
    s = q;
    std::cout << "Copy Assigned Array: \n";
    s.DisplayAll();

    std::cout << "Dequeued: " << q.Dequeue() << std::endl;
    q.DisplayAll();
    std::cout << "Dequeued: " << q.Dequeue() << std::endl;
    q.DisplayAll();
    std::cout << "Dequeued: " << q.Dequeue() << std::endl;
    q.DisplayAll();
    std::cout << "Dequeued: " << q.Dequeue() << std::endl;
    q.DisplayAll();
    std::cout << "Dequeued: " << q.Dequeue() << std::endl;
    q.DisplayAll();
    std::cout << "Is queue empty? " << ((q.Empty()) ? "Yes" : "No") << std::endl;

    return 0;
}
```

```
Enqueueing: ROYCE
Front: R
Back: E
Queue: ROYCE
Copied Array:
ROYCE
Copy Assigned Array:
ROYCE
Dequeued: R
OYCE
Dequeued: O
YCE
Dequeued: Y
CE
Dequeued: C
E
Dequeued: E
Queue is empty
Is queue empty? Yes
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using queue implementation

Problem Title: Shared Printer Simulation using Queues

Problem Definition: In this activity, we'll simulate a queue for a shared printer in an office. In any corporate office, usually, the printer is shared across the whole floor in the printer room. All the computers in this room are connected to the same printer. But a printer can do only one printing job at any point in time, and it also takes some time to complete any job. In the meantime, some other user can send another print request. In such a case, a printer needs to store all the pending jobs somewhere so that it can take them up once its current task is done.

Perform the following steps to solve the activity. Make sure that you include a screenshot of the source code for each. From the get-go: You must choose whether you are making a linked list or array implementation. You may NOT use the STL.

1. Create a class called Job (comprising an ID for the job, the name of the user who submitted it, and the number of pages).

```cpp
class Job {
    private:
        int Id;
        std::string Name;
        int Pages;

    public:
        Job(int id = 0, std::string name = "", int pages = 0) {
            Id = id;
            Name = name;
            Pages = pages;
        }

        int getID() {
            return Id;
        }
        std::string getName() {
            return Name;
        }
        int getPages() {
            return Pages;
        }

        void PrintDetails() {
            std::cout << "ID: " << Id << std::endl;
            std::cout << "Name: " << Name << std::endl;
            std::cout << "Page Count: " << Pages << std::endl << std::endl;
        }
};
```

- *This class stores the three important details of a job, and has basic getter functions.*

2. Create a class called Printer. This will provide an interface to add new jobs and process all the jobs added so far.

```cpp
class Printer {
    private:
        qNode* firstJob;
        qNode* lastJob;
    public:
        Printer() {
            firstJob = nullptr;
            lastJob = nullptr;
        }

        ~Printer() {
            while(firstJob != nullptr) {
                qNode* temp = firstJob;
                firstJob = firstJob->next;
                delete temp;
            }
            lastJob = nullptr;
        }

        bool Empty() {
            return (firstJob == nullptr && lastJob == nullptr);
        }
```

- *This printer class uses the qNode class so that we can put it into a linked list*

3. To implement the printer class, it will need to store all the pending jobs. We'll implement a very basic strategy – first come, first served. Whoever submits the job first will be the first to get the job done.

```
void addJob(Job newJob) {
    qNode* newNode = new qNode(newJob);

    if(Empty()) {
        firstJob = newNode;
        lastJob = newNode;
    } else {
        lastJob->next = newNode;
        lastJob = newNode;
    }

    std::cout << "Added!" << std::endl;
    newJob.PrintDetails();
}
```

```
void StartPrinting() {
    std::cout << "PRINTING_ _ _" << std::endl;

    if(Empty()) {
        std::cout << "There was nothing to print." << std::endl;
        return;
    }

    while(firstJob != nullptr) {
        qNode* temp = firstJob;

        Sleep(1000 * temp->data.getPages());

        std::cout << "Printed: " << temp->data.getName() << std::endl;


        if(firstJob == lastJob) {
            firstJob = nullptr;
            lastJob = nullptr;
        } else {
            firstJob = firstJob->next;
        }

        temp->next = nullptr;
        delete temp;
    }

    std::cout << "Printing Finished!!!" << std::endl << std::endl;
}
```

- addJob is a basic addNode, and the Printing is a traverse function for linked lists. I added an additional Sleep function to simulate time passing as the pages get printed.

4. Finally, simulate a scenario where multiple people are adding jobs to the printer, and the printer is processing them one by one.

```
int main() {
    //4. Finally, simulate a scenario where
    Printer printer1;

    Job print1(1, "Resume", 1);
    Job print2(2, "Research", 15);
    Job print3(3, "Report Cards", 5);

    printer1.addJob(print1);
    printer1.addJob(print2);
    printer1.addJob(print3);

    printer1.StartPrinting();

    Job print4(4, "Assignment", 2);
    printer1.addJob(print4);
    printer1.StartPrinting();

    return 0;
}
```

5. Defend your choice of internal representation: Why did you use arrays or linked lists?

- I personally chose the linked list implementation since it is the easier method of implementing queues. A linked list already works like a queue so I didn't have to think much of the constructor, printing, and deconstructor functions.

Output Analysis:

● Provide the output after performing each task above.

```
Added!
ID: 1
Name: Resume
Page Count: 1

Added!
ID: 2
Name: Research
Page Count: 15

Added!
ID: 3
Name: Report Cards
Page Count: 5

PRINTING_ _ _
Printed: Resume
Printed: Research
Printed: Report Cards
Printing Finished!!!

Added!
ID: 4
Name: Assignment
Page Count: 2

PRINTING_ _ _
Printed: Assignment
Printing Finished!!!
```

● Include your analysis: focus on why you think the output is the way it is

- The output was successful and the time while printing was also simulated properly.

3. CONCLUSIONS/LEARNINGS

- This Hands on activity helped me understand deeply how a queue works from using the standard library, to creating my own prototype from linked list implementation, to array implementation, and even creating a custom Printer queue maker.

4. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: **Error fixing, additional functions.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The AI tool was used to pinpoint the errors in the code and to learn the Sleep function to simulate time passing**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.